CS 498:
Adaptive KV Cache

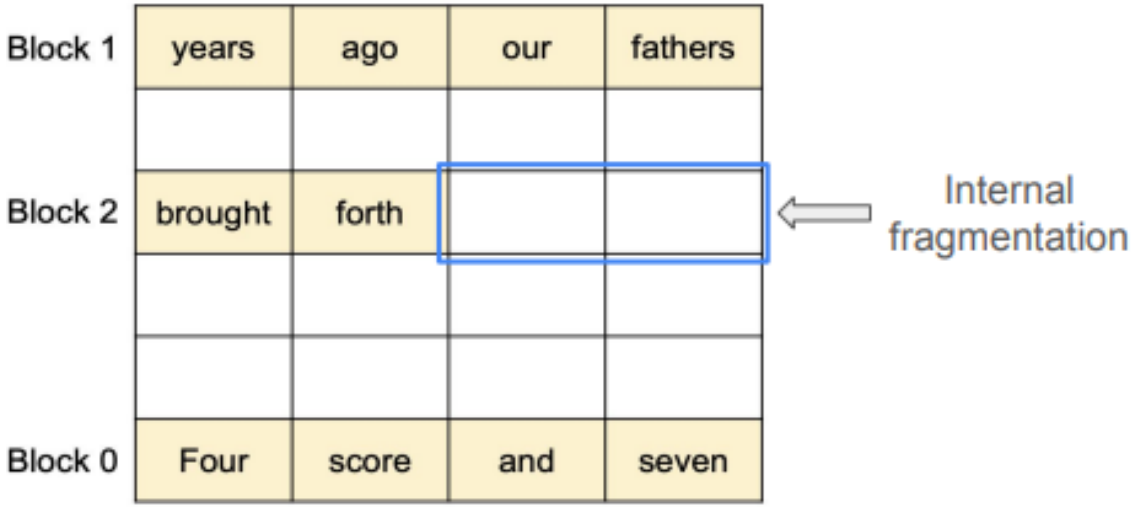
Fan Lai

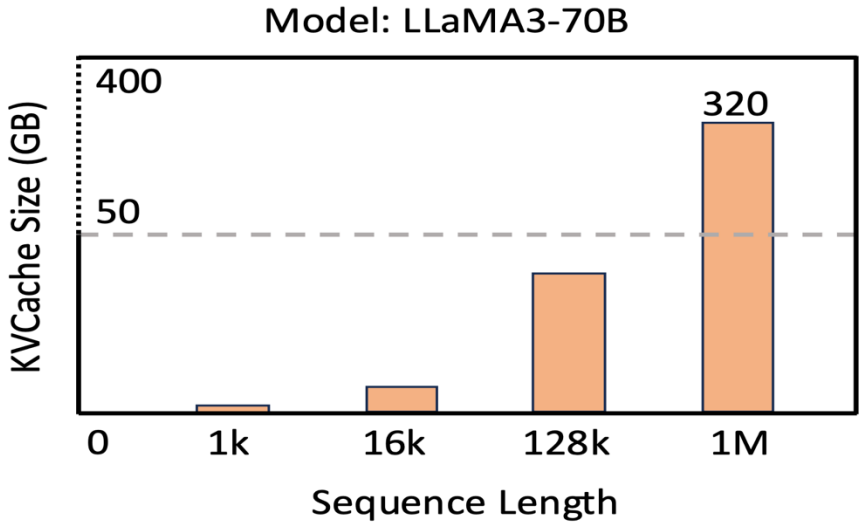
The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

Recall: Memory Efficiency of PagedAttention

- Minimal internal fragmentation
  - # of wasted tokens per sequence < block size
- No external fragmentation





KV Cache introduces great memory challenges

Today

LLM Inference with smaller KV cache

- H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs
- Attention Sink: Efficient Streaming LLMs with Attention Sinks

Objective: Explain the motivation and techniques of adaptive KV cache, using H2O and Attention Sink as case studies

Motivation for Adaptive Caching

- Full KV cache with paged attention is memory-efficient, but not content-aware.

- Can we evict or compress KV pairs that are unlikely to influence future predictions?

Challenges with Cache Eviction

- Ideal KV Cache?
  - i. Small cache size to reduce memory footprint
  - ii. Low miss rate
  - iii. Low-cost eviction policy

Challenges with Cache Eviction

- Ideal KV Cache
  - i. Small cache size to reduce memory footprint
    - Challenge: Whether the size of KV cache can be restricted?
  - ii. Low miss rate
  - iii. Low-cost eviction policy

Challenges with Cache Eviction

- Ideal KV Cache
  - i. Small cache size to reduce memory footprint
    - Challenge: Whether the size of KV cache can be restricted?
  - ii. Low miss rate
    - Challenge: Evicted KV embeddings can not be accessed in the future. How to maintain the performance and long-content generation accuracy?
  - iii. Low-cost eviction policy

Challenges with Cache Eviction

- Ideal KV Cache
  - i. Small cache size to reduce memory footprint
    - Challenge: Whether the size of KV cache can be restricted?
  - ii. Low miss rate
    - Challenge: Evicted KV embeddings can not be accessed in the future. How to maintain the performance and long-content generation accuracy?
  - iii. Low-cost eviction policy
    - Challenge: Combination problem, brute force policy might lead to high latency

H2O Observation 1

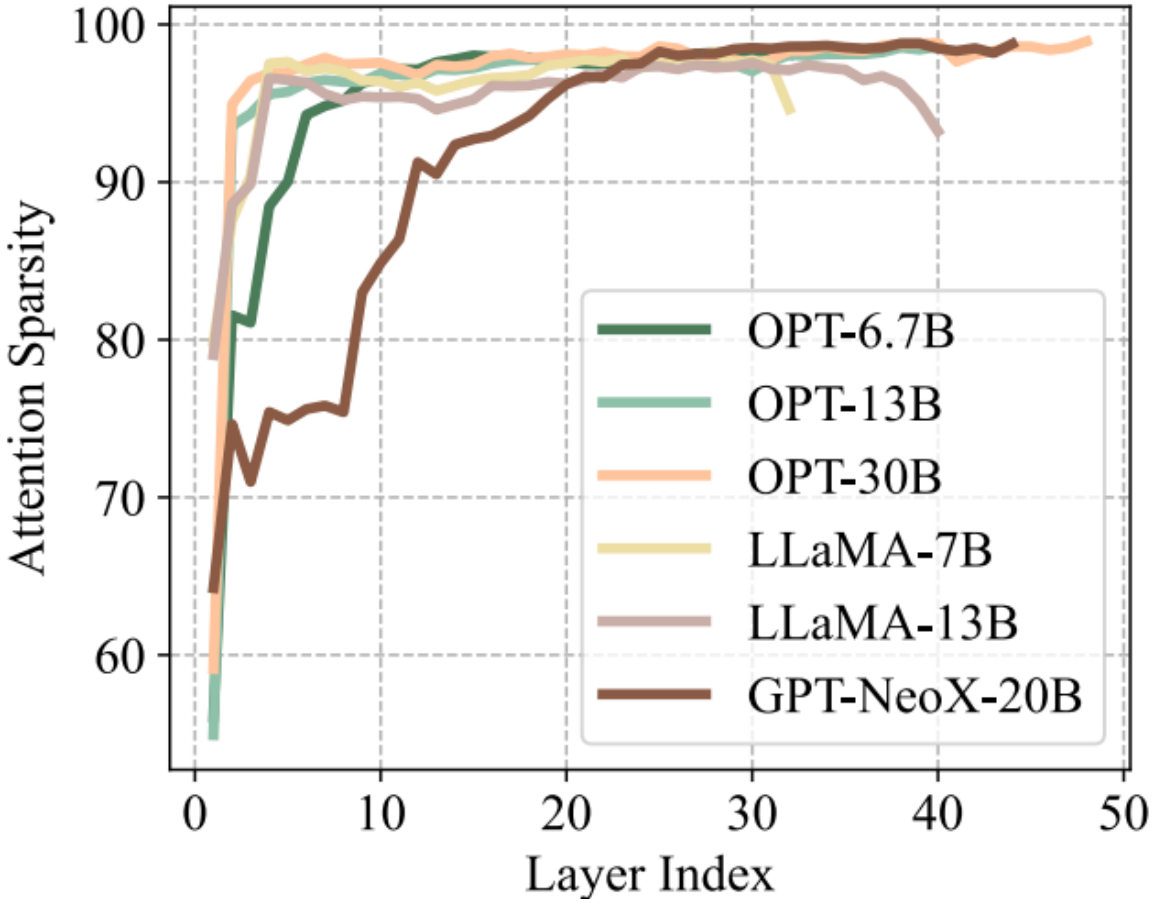
- Attention Sparsity

  - Threshold = 1% x Maximum attention score from softmax (QK^T)

$$\text{Attention score} = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})$$

H2O Observation 1

- Sparsity for small cache size



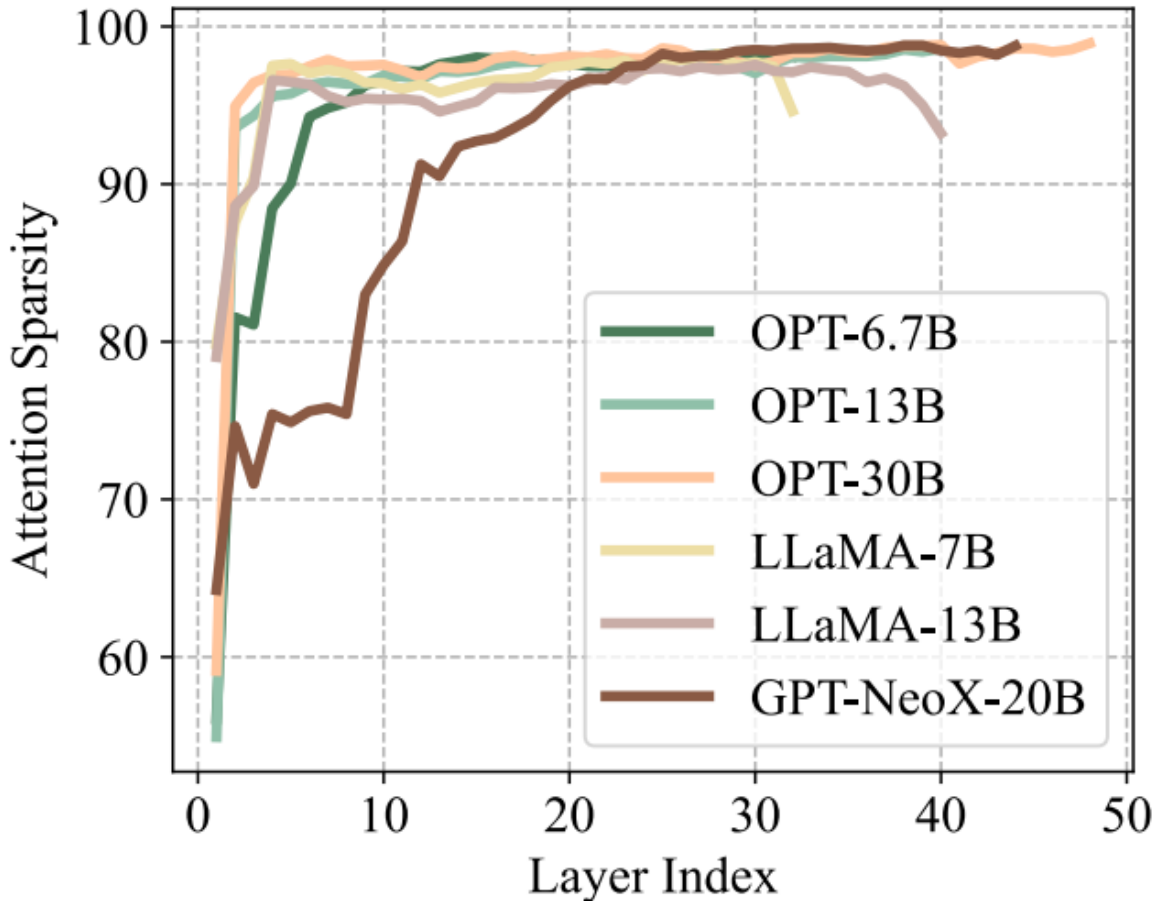
- Attention Sparsity

  - Threshold = 1% x Maximum attention score from softmax (QK^T)

- Sparsity over 95% in almost all layers

H2O Observation 1

- Sparsity for small cache size



- Attention Sparsity

  - Threshold = 1% x Maximum attention score from softmax (QK^T)

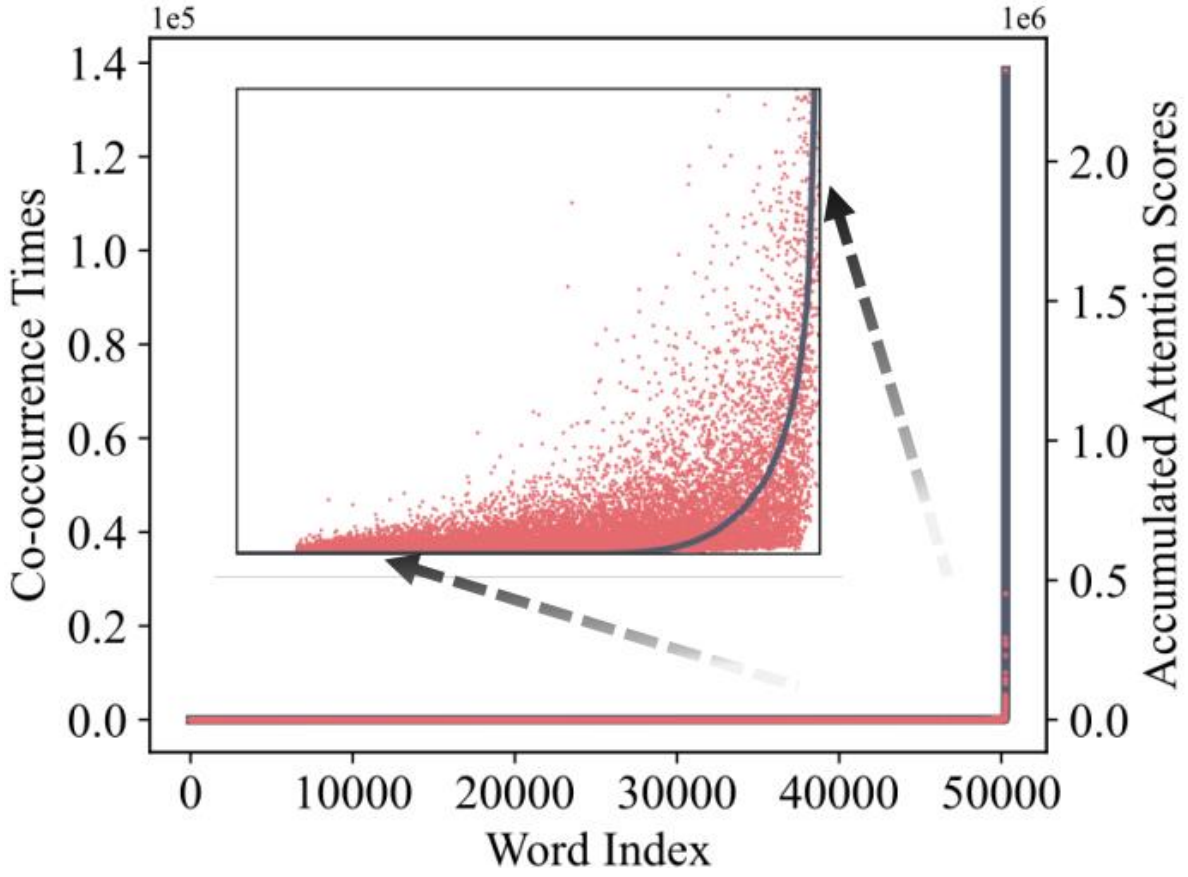
- Sparsity over 95% in almost all layers

- Access to all previous key, value is unnecessary for generating the next token

Takeaway 1: Perhaps a small cache size is possible. But which tokens should stay in cache?

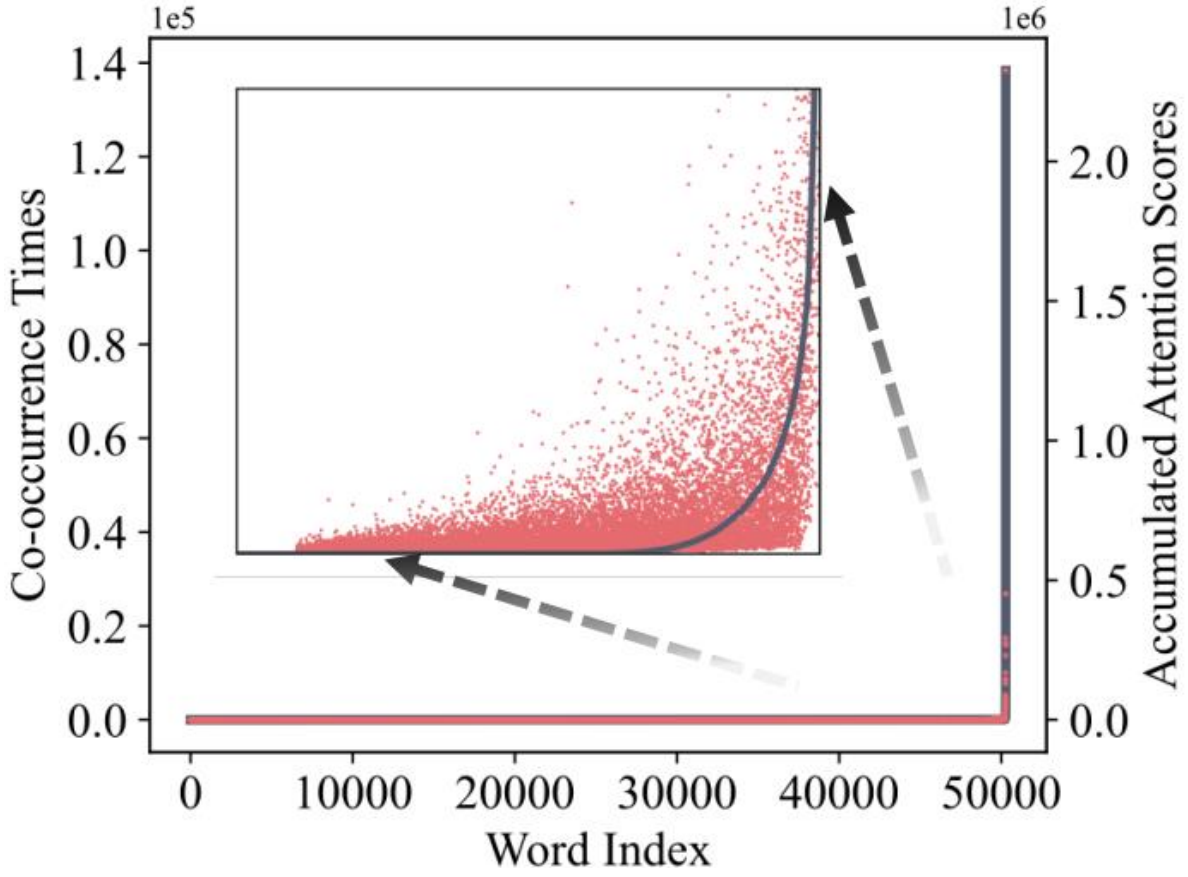
Observation 2

- Heavy-Hitters for Low Miss Rate



Observation 2

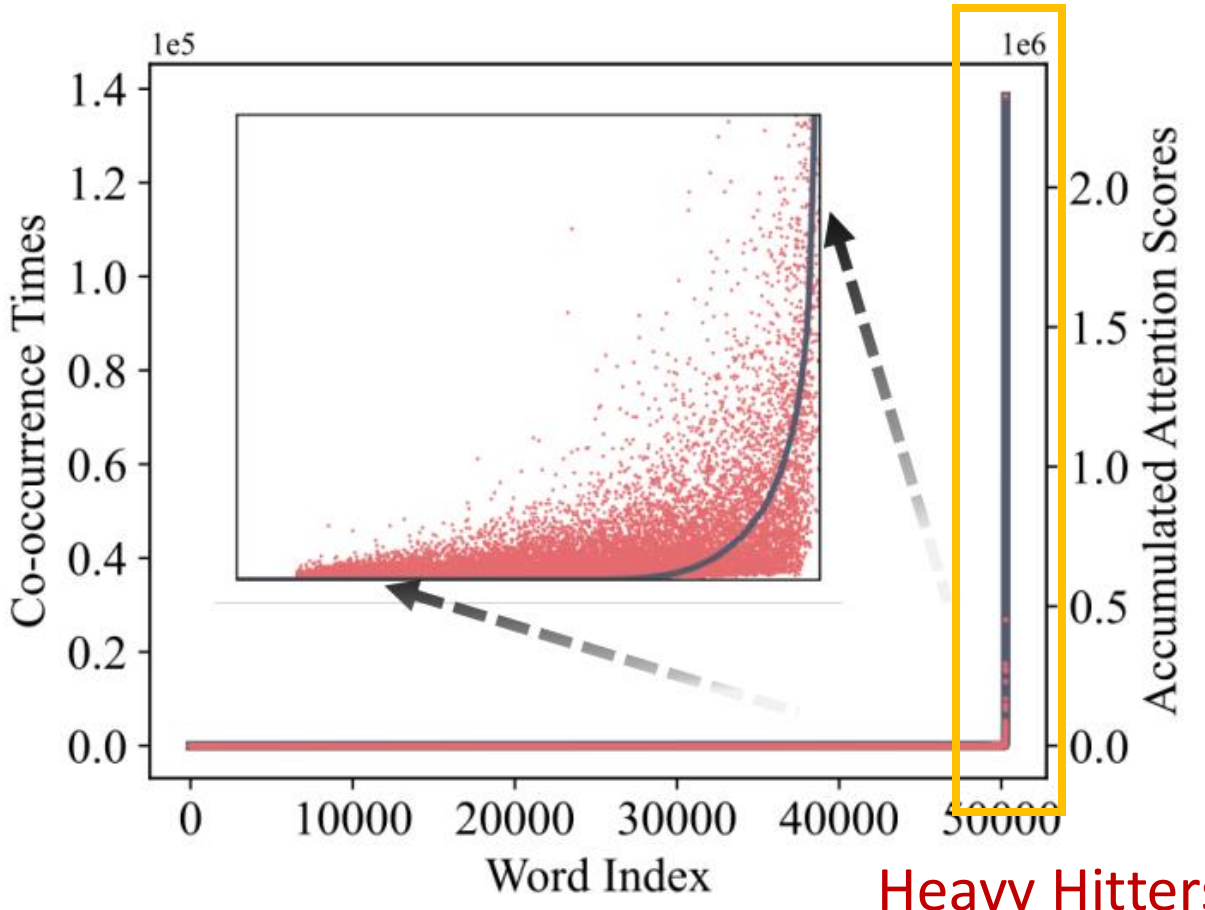
- Heavy-Hitters for Low Miss Rate



- Power-law distribution
  - Accumulated attention score (red)
  - A small set of tokens are critical during generation
  - High correlation with co-occurrences (gray) in the data

Observation 2

- Heavy-Hitters for Low Miss Rate

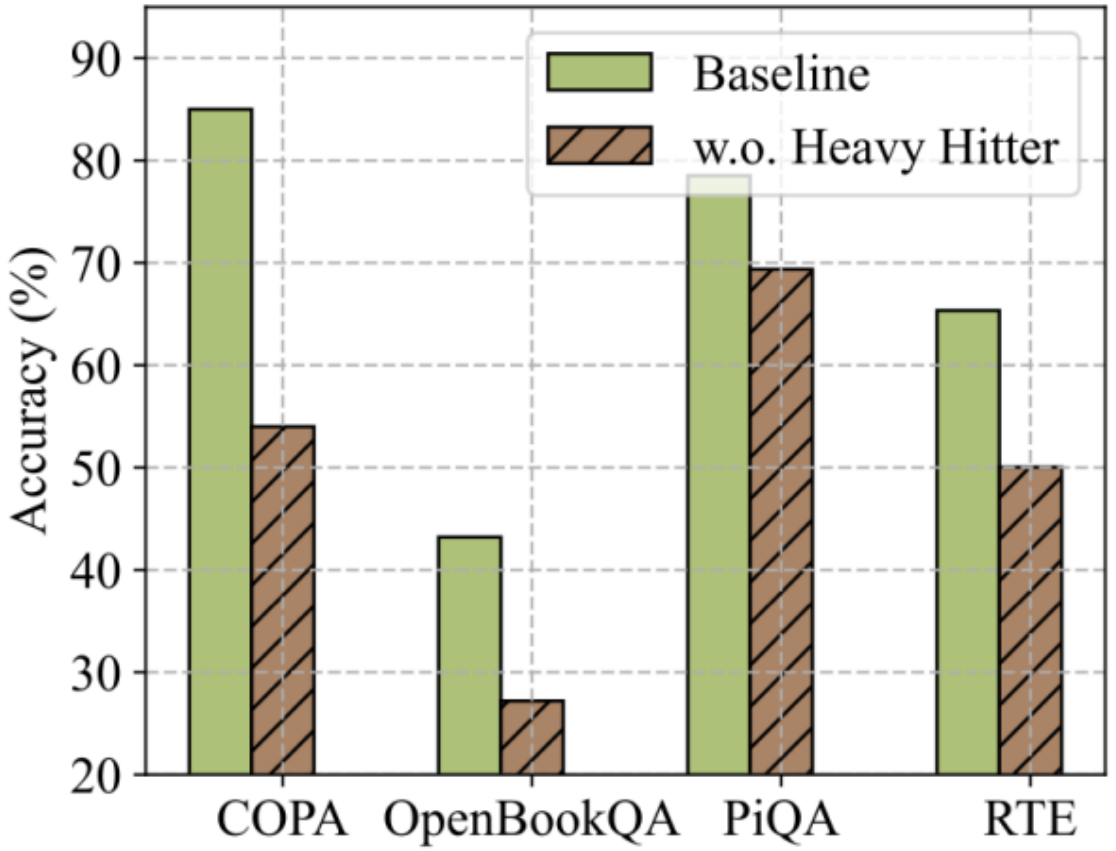


- Power-law distribution
  - Accumulated attention score (red)
  - A small set of tokens are critical during generation
  - High correlation with co-occurrences (gray) in the data

Takeaway 2: Aggregated attention score might be a good signal for determining tokens to retain in KV cache

Observation 3

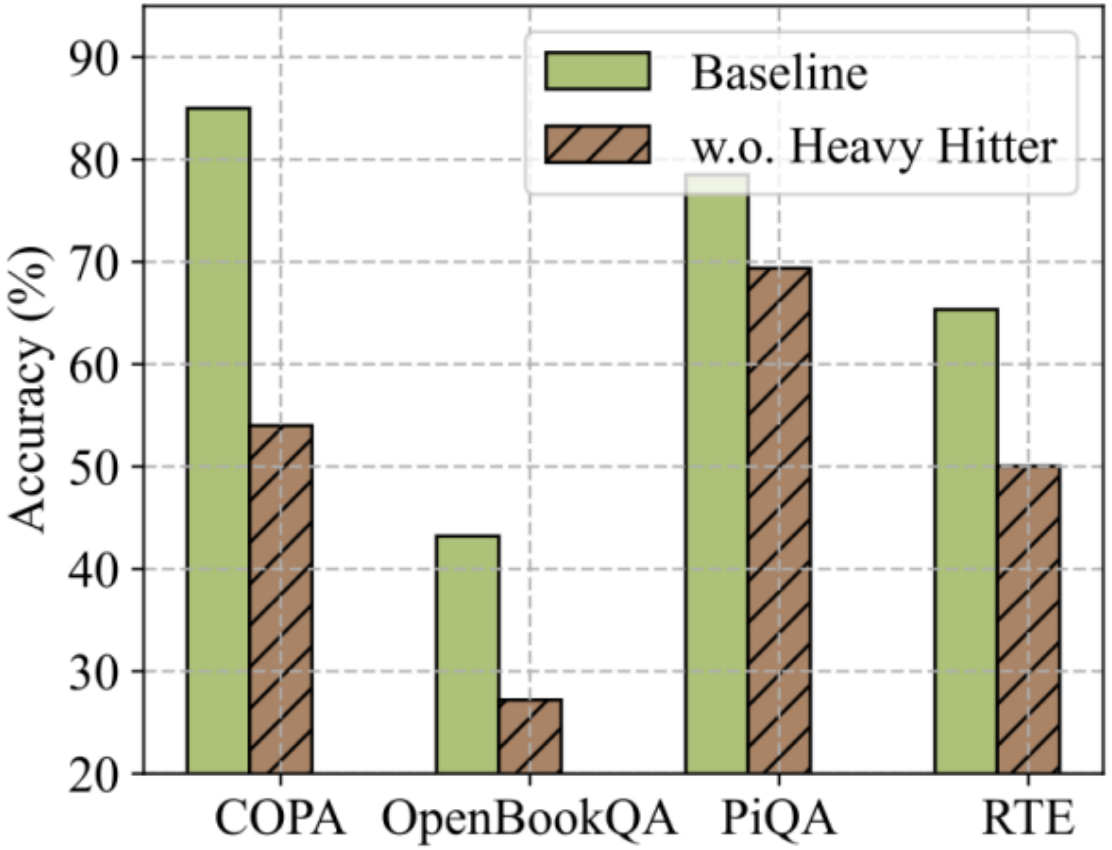
- A small portion of tokens contribute most to attention score



- Evicting Heavy-Hitters destroy the performance of LLMs

Problem Formulation

Formulate KV cache eviction as a knapsack packing problem and use a greedy policy for cache eviction



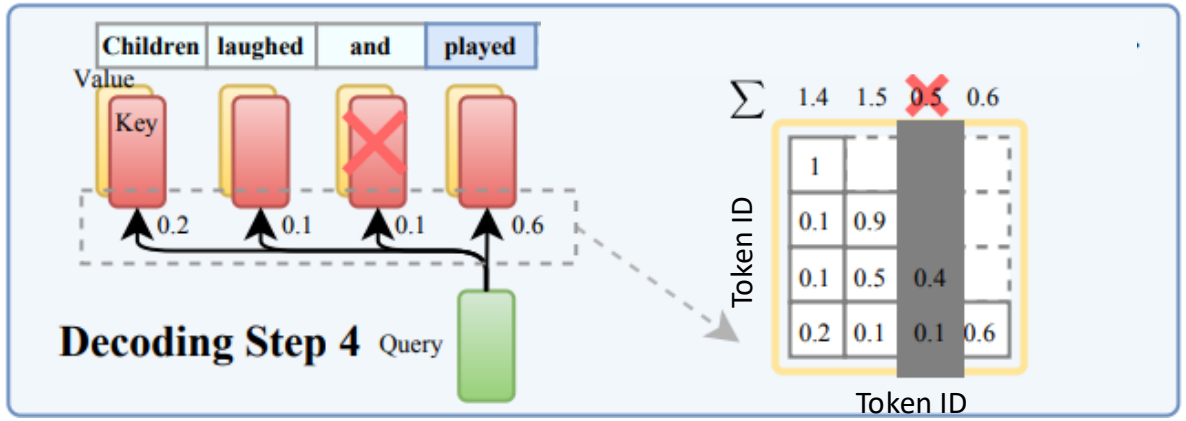
An optimization problem: the goal is to select a subset of items (from a large set) that satisfies certain constraints (e.g., budget) while maximizing the total item value (e.g., benefit)

Heavy-Hitter Oracle

- Greedy Algorithm for Low-Cost Policy
  - Keep top-K tokens with highest aggregated attention scores

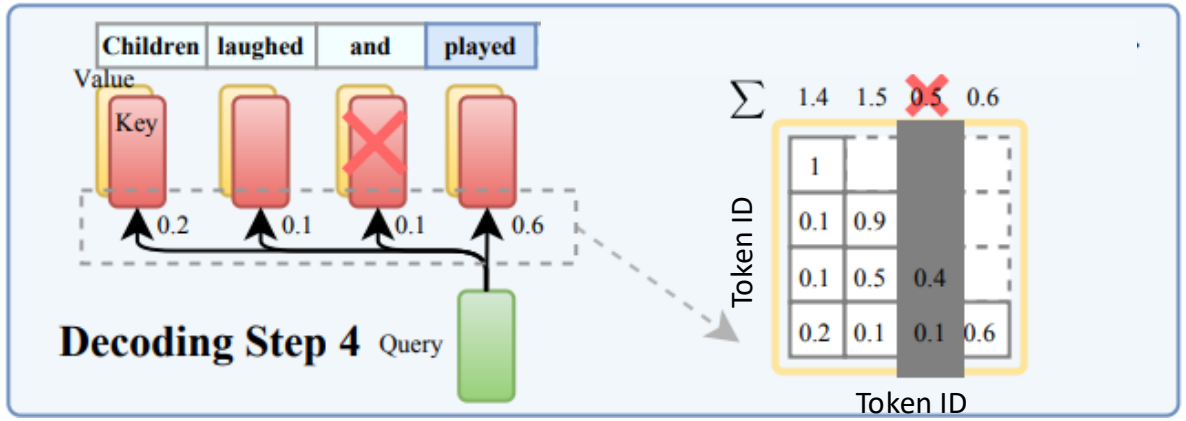
Heavy-Hitter Oracle

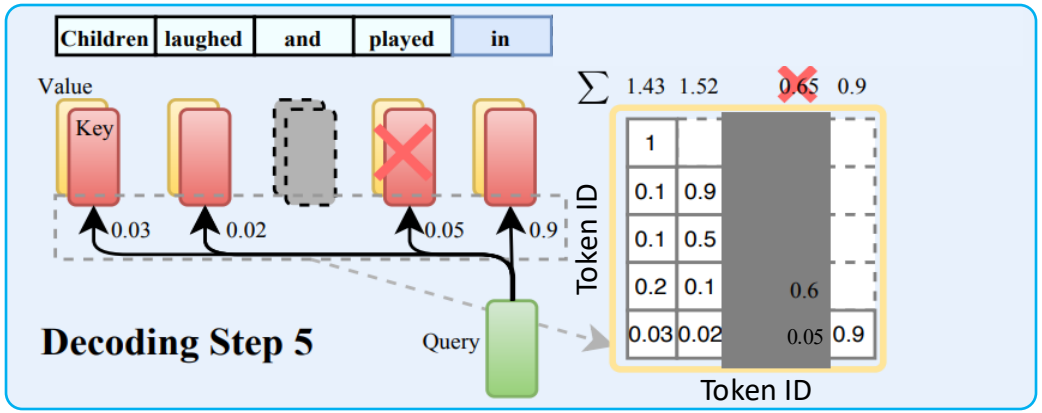
- Greedy Algorithm for Low-Cost Policy
  - Keep top-K tokens with highest aggregated attention scores



Heavy-Hitter Oracle

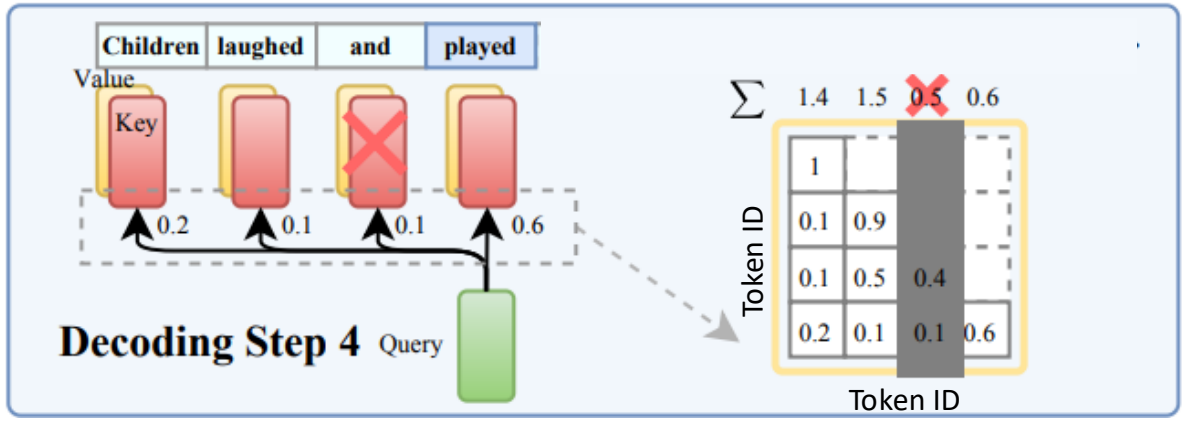
- Greedy Algorithm for Low-Cost Policy
  - Keep top-K tokens with highest aggregated attention scores

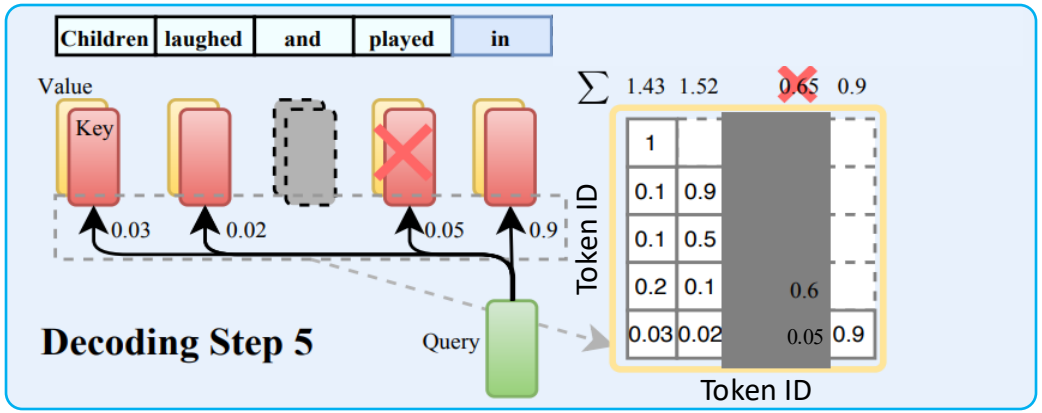




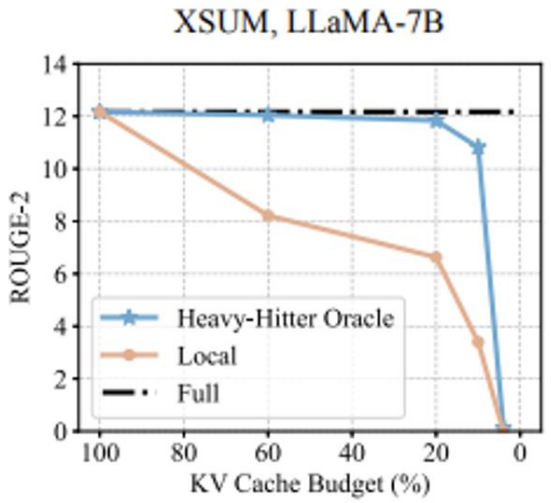
Heavy-Hitter Oracle

- Greedy Algorithm for Low-Cost Policy
  - Keep top-K tokens with highest aggregated attention scores

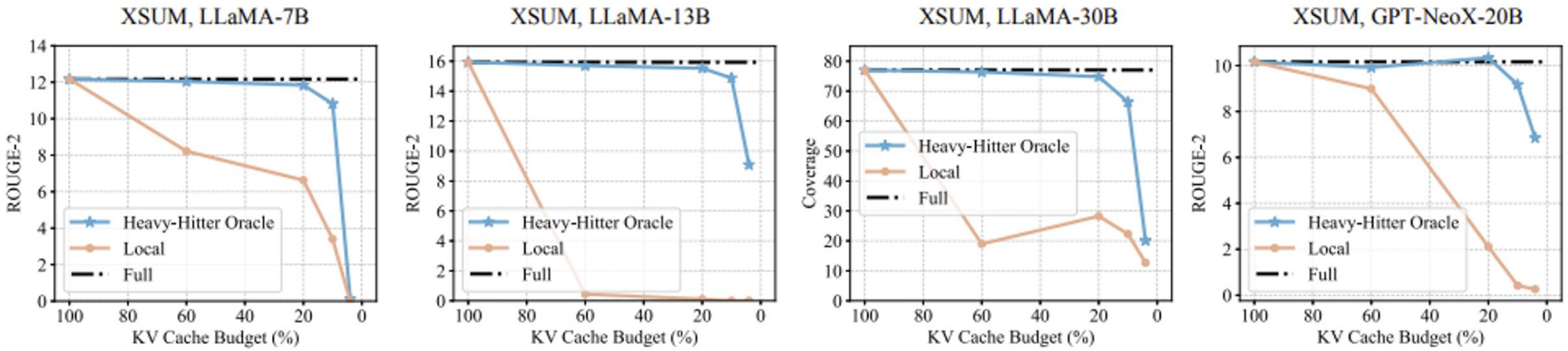




Heavy-Hitter Oracle Achieves Good Compression



Heavy-Hitter Oracle Achieves Good Compression



Drawbacks of the H2O design?

Attention Sink

Key observation

- Most of the attention score is held in the first few “sink” tokens
  - Preserve these tokens and reuse the KV cache for last L tokens
  - O(L) computation without much loss in performance

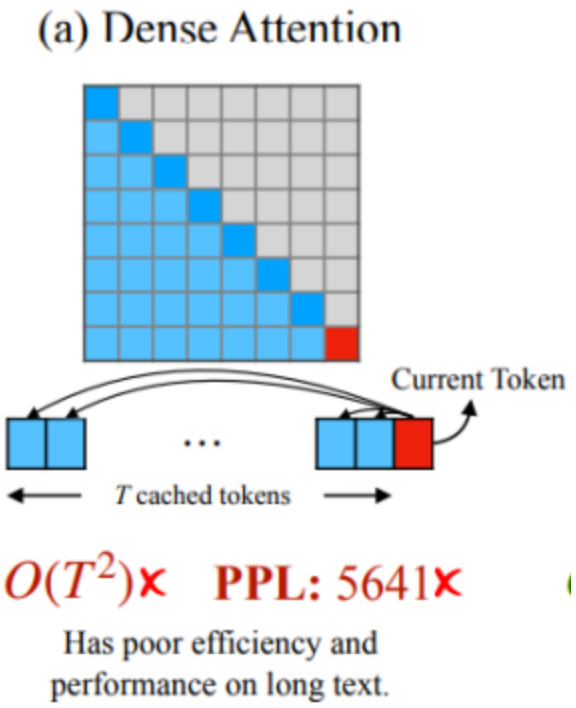
EFFICIENT STREAMING LANGUAGE MODELS WITH ATTENTION SINKS

Guangxuan Xiao[1*] **Yuandong Tian**[2] **Beidi Chen**[3] **Song Han**[1,4] **Mike Lewis**[2]

[1] Massachusetts Institute of Technology
[2] Meta AI
[3] Carnegie Mellon University
[4] NVIDIA

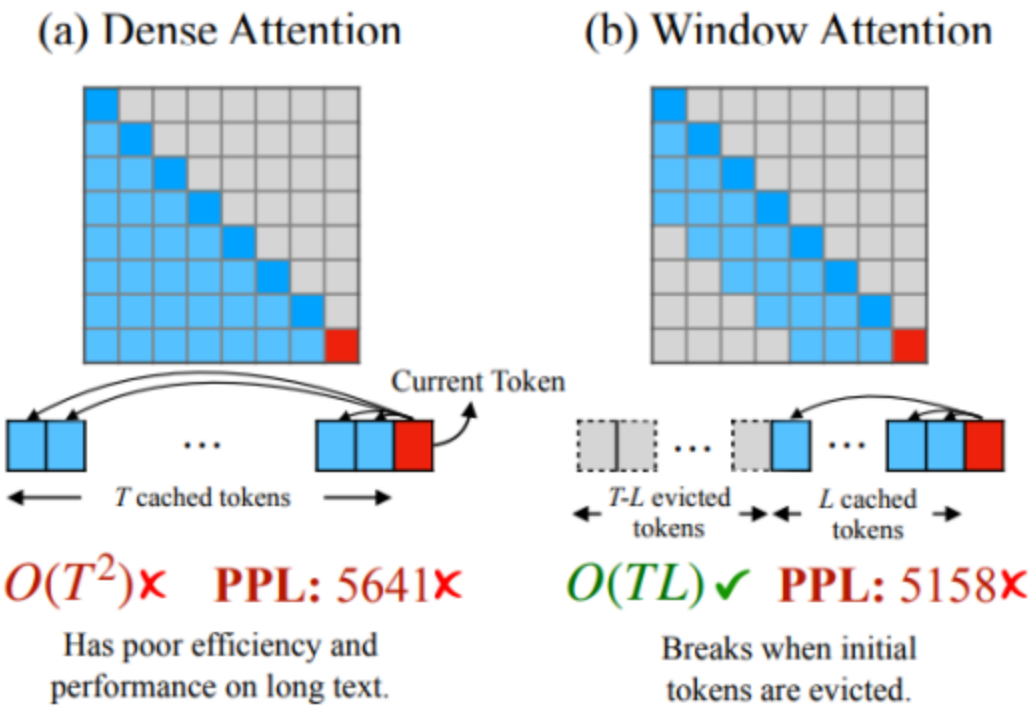
https://github.com/mit-han-lab/streaming-llm

Attention Sink



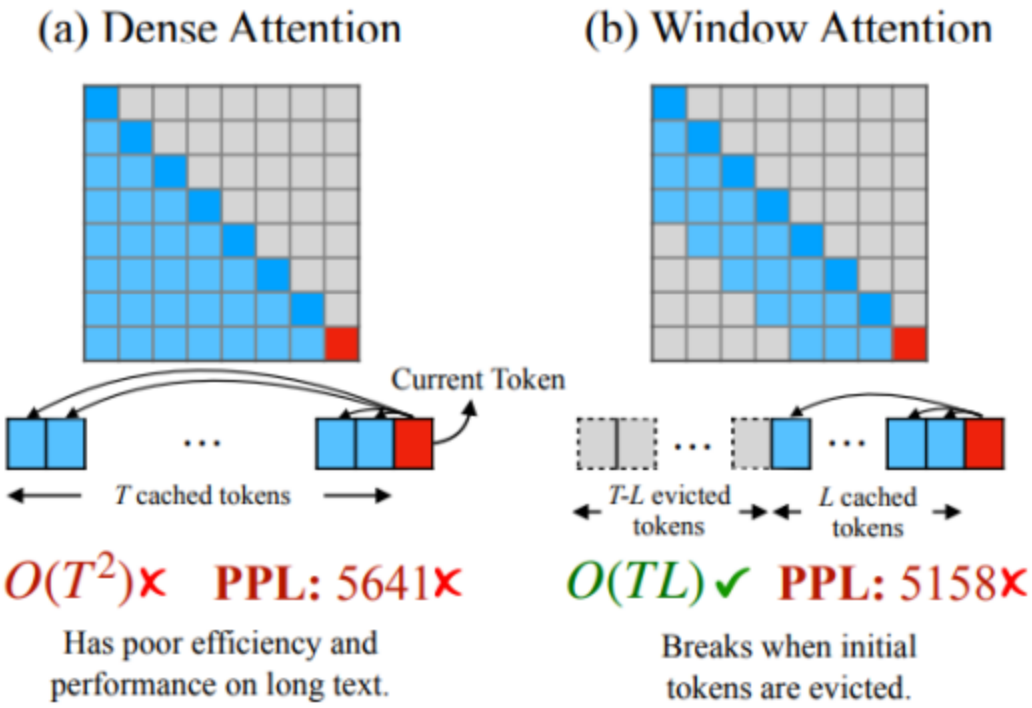
Lower perplexity (PPL) indicates better response quality.

Attention Sink

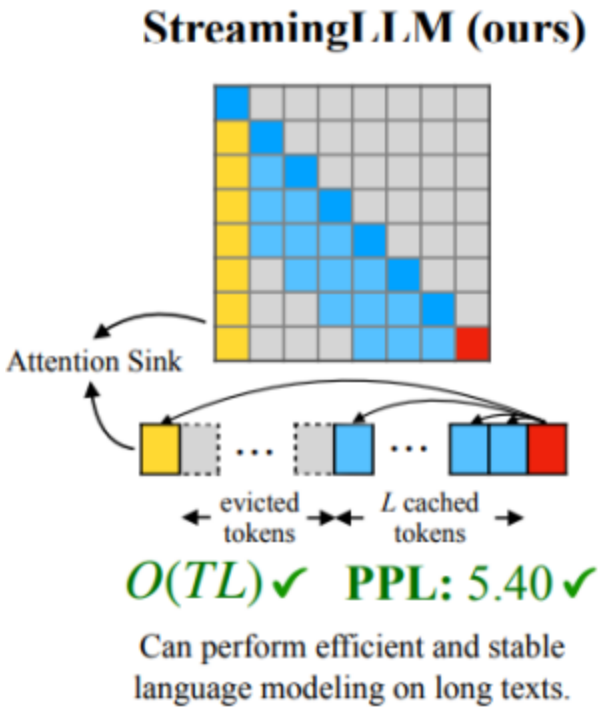


(better PPL due to lost-in-the-middle issue)

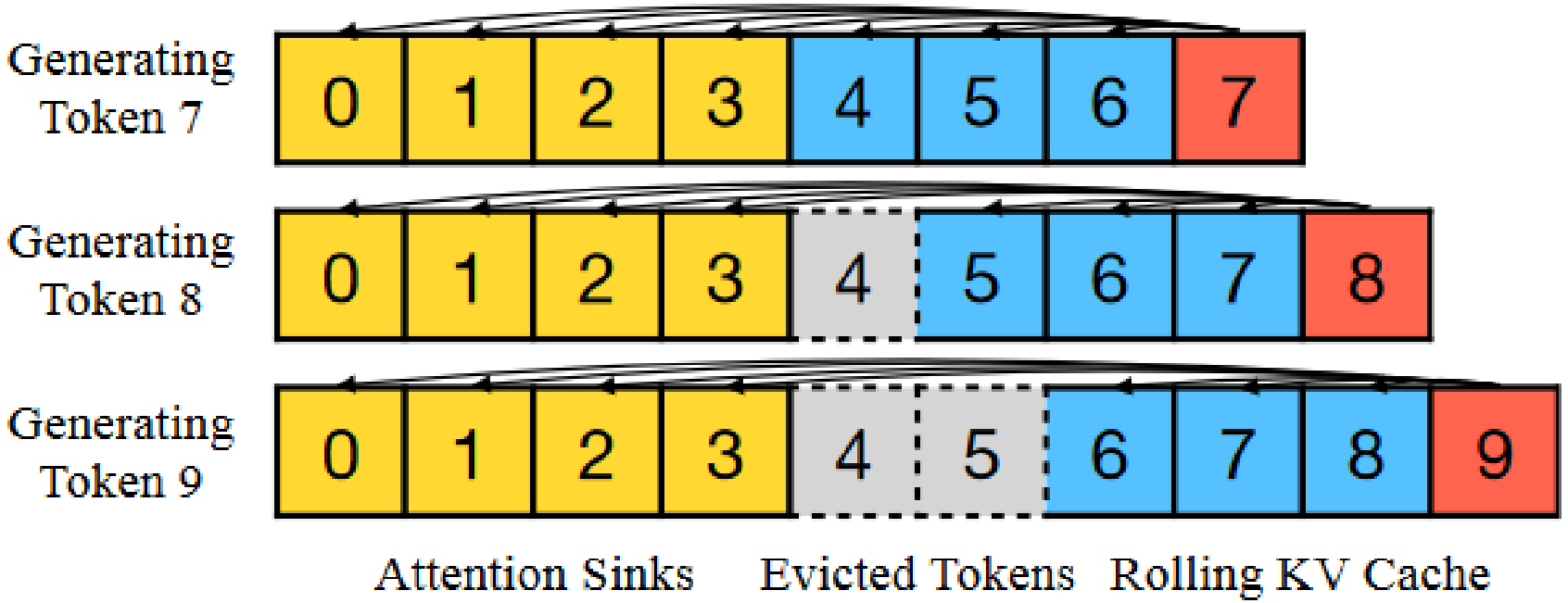
Attention Sink



(better PPL due to lost-in-the-middle issue)



StreamingLLM: Attention Sink + Window Attention



1. Always keep the initial few tokens, i.e., the attention sink tokens
2. Rolling window

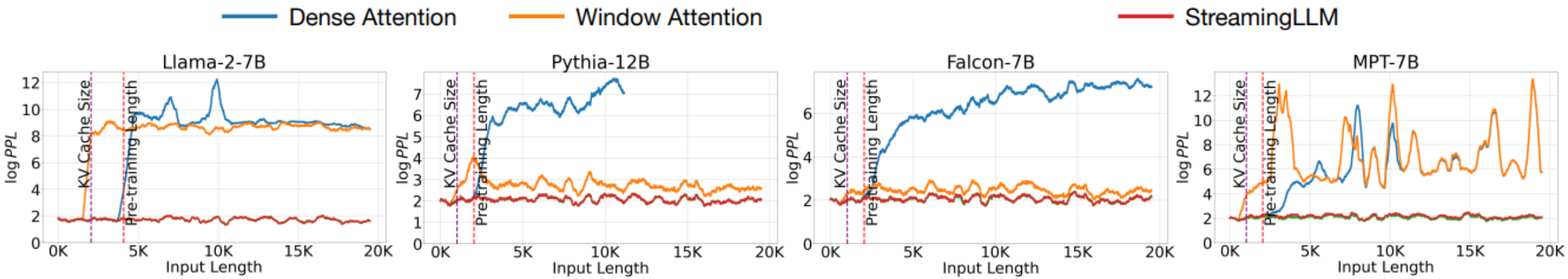
Attention Sink – Perplexity Experiments

Lower perplexity (PPL) indicates better response quality.

- Dense attention fails once the input length surpasses the pre-training attention window size
- Window attention collapses once the input length exceeds the cache size

Attention Sink – Perplexity Experiments

Lower perplexity (PPL) indicates better response quality.



- Dense attention fails once the input length surpasses the pre-training attention window size
- Window attention collapses once the input length exceeds the cache size

Summary

- H2O: not all KV cache (tokens) are equally important
  - Keep the ones with high attention score to cap memory usage

- Attention Sink:
  - Keep the first several tokens and recent tokens
  - Hard to interpret

Assignment 2 Released (Due on Nov 19)

- Short questions
  - LoRA, PagedAttention, SGLang, …
- Lab Assignments
  - Multi-head Attention
  - Grouped-query Attention
  - Mixture of expert (MoE) models